

Atlas — Complete Documentation

Multi-tenant SaaS for travel agencies

Generated 2026-06-28 · 16 documents

- Live: <https://agencetool.vercel.app>
- Repo: github.com/ouks1993/agence_tool

Atlas — Documentation Index

Atlas is the operating system for modern travel agencies. Every decision should support that sentence.

Atlas is a **multi-tenant SaaS for travel agencies**. Each agency runs its clients, sales pipeline, proposals, bookings and finance in a fully isolated workspace; the vendor manages every agency from a platform console. Multilingual (EN/FR/AR with RTL), deployed on Vercel with GitHub auto-deploy.

- **Live:** <https://agencetool.vercel.app>
- **Repo:** github.com/ouks1993/agence_tool
- **Vendor console:** <https://agencetool.vercel.app/platform>

This is the reference index. The detailed reference is split into focused docs under `docs/`. See also `DESIGN.md` (design system) and `AGENTS.md/CLAUDE.md` (working conventions) — those remain the source of truth for UI and coding conventions respectively.

Documentation map

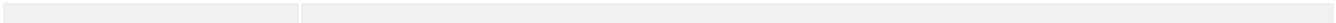
Doc	Covers
Vision	What Atlas is, who it's for, operating model, features at a glance
Architecture	Multi-tenancy · auth · platform admin · impersonation · roles · route map · module layout · i18n
Tech stack	Framework, libraries, services
Domain model	Core entities, the Lead→...→Feedback chain, aggregates & ownership
Database	Schema tables, tenancy columns, migrations, ID convention
API integrations	Duffel · Hotelbeds · Stripe (billing + Connect) · Resend · OpenRouter · Blob
Development guide	Setup, env vars, commands, scripts, DB workflow
Coding standards	→ points to <code>AGENTS.md</code> ; conventions summary
UI / UX	→ points to <code>DESIGN.md</code> ; product UX patterns
Business rules	RBAC, currency, booking lifecycle, commissions, vocabularies
Deployment	Vercel, env, prod migrations, demo accounts
Roadmap & changelog	Phase status, open items, commit changelog
Security	Tenant isolation, auth, guards, webhooks, DB safety, known risks
Analytics & BI	Dashboards, currency-safe metrics, CSV/Excel export

AI	Assistant + inline AI features
----	--------------------------------

At a glance

- **Stack:** Next.js 16 (App Router) · React 19 · TypeScript · Tailwind v4 + shadcn/ui · PostgreSQL (Neon) + Drizzle · Better Auth · next-intl · Stripe + Stripe Connect · Resend · Duffel (flights) · Hotelbeds (hotels) · Vercel AI SDK + OpenRouter · Vercel.
- **Tenancy:** every business row is scoped to an `agencyId`; enforced via `requireAgencyUser()`. Vendor platform admin sits above all tenants.
- **Roles:** admin · manager · finance · support · agent — each with a tailored landing and nav.
- **Deploy:** push to `main` → Vercel auto-deploys production.
- **DBs:** dev `ep-wandering-sunset-aitlty78` · prod `ep-misty-thunder-aixz34vy`. Run `POSTGRES_URL=<prod-url> npx drizzle-kit migrate` after each schema change.

Phases 1–3 + UX + Data-quality/BI complete. Multi-tenant SaaS with live travel sourcing, billing, client portal, supplier/commission management, and inline AI features. Migrations: 18 (latest 0018).



Vision

Atlas is the operating system for modern travel agencies.

Every decision should support that sentence.

It is the north star: a feature, a design choice, or a piece of scope earns its place only if it makes Atlas more of an agency's single operating system. If it doesn't support the sentence, it doesn't ship.

Today an agency runs on a patchwork of tools:

- Excel
- WhatsApp
- Email
- Booking portals
- Accounting software

Everything should happen inside Atlas instead — **one workspace**.

Long-term goals

- CRM
- Sales
- Quotations
- Flight booking
- Hotel booking
- Supplier management
- Accounting
- Customer portal
- AI assistant
- BI
- Automation

Operating model

- **Per-agency isolation.** Every business record is scoped to an `agencyId`; agencies never see each other's data. Reference numbers (`BKG-...`, `PRD-...`) are unique per agency. See [architecture.md](#) and [security.md](#).
- **Vendor + tenants.** The vendor (platform operator) sells Atlas to many agencies and bills them by subscription, managing all of them from a platform console. Each agency is a fully isolated tenant.
- **DZD-first, no FX.** The agency operates in Algerian Dinar. EUR/USD are supported for source data but Atlas never converts between currencies — analytics group by currency rather than summing across. See [business-rules.md](#).

- **Graceful degradation.** Every external integration (flights, hotels, email, payments, AI) falls back to sample/logged behaviour when its keys are unset, so the app runs end-to-end with only a database and an auth secret. See [api-integrations.md](#).

Where it is today

Atlas is a deployed, multi-tenant SaaS — multilingual (EN/FR/AR with RTL), live on Vercel with GitHub auto-deploy.

- **Live:** <https://agencetool.vercel.app>
- **Repo:** github.com/ouks1993/agence_tool
- **Vendor console:** <https://agencetool.vercel.app/platform>

Shipped against the long-term goals:

- **CRM** — clients with contacts and a unified funnel timeline.
- **Sales** — opportunities, pipeline stages, conversion analytics.
- **Quotations** — proposals with PDF + e-signature, one-click convert→booking, AI quote builder.
- **Flight booking** — live Duffel search (real order placement is pending).
- **Hotel booking** — live Hotelbeds search + content cache (real booking pending).
- **Supplier management** — directory, contracts & rates, supplier picker.
- **Customer portal** — magic-link login, trip view, online payments, proposal signing.
- **AI assistant** — agency-scoped chat + inline itinerary/quote/email/visa features. See [ai.md](#).
- **BI** — decision dashboards + standardized CSV/Excel export. See [analytics.md](#).
- **Accounting** — partial: payments, two-ledger commissions, AR aging. Full accounting is a long-term goal.
- **Automation** — early: auto-generated commissions, lifecycle guards. Broader automation (e.g. quote-on-stage-change, WhatsApp) is on the roadmap.

Atlas principles

The ten founding principles. Everything else — the design system, the never-rules, the golden workflow — is an expression of these.

1. **Client first.** The client record is the spine; every workflow starts there.
2. **Every entity belongs to an agency.** Multi-tenancy is non-negotiable — `agencyId` on every business row, enforced on every read/write.
3. **No duplicated data.** One source of truth; canonical values over free text.
4. **Automation over manual work.** Generate, don't ask — quotes, itineraries, commissions, documents.
5. **Every workflow has one happy path.** A single, obvious golden route through each flow ([business-rules.md](#)).
6. **Managers need insight.** Decision-oriented analytics where managers land.
7. **Agents need speed.** Fast, scoped surfaces — minimal clicks, minimal waiting.
8. **Data quality > flexibility.** Constrain inputs to keep reporting clean.
9. **Everything is reversible.** Prefer soft state and undo over destructive operations.

10. **Every page answers "What do I do next?"** Always surface the next action; never a dead end.

These are operationalized as UI guardrails in `ui-ux.md`, hard constraints in `business-rules.md`, and tracked against the code in the gap tracker.

See `roadmap.md` for phase status and open items, and the index for the full documentation map.

Tech Stack

Layer	Choice
Framework	Next.js 16 (App Router), React 19, TypeScript
Styling	Tailwind CSS v4 (<code>@theme inline</code>), shadcn/ui (new-york), Lucide icons
Fonts	Geist (sans/mono), IBM Plex Sans Arabic (RTL)
Auth	Better Auth (email/password, invitation-gated)
Database	PostgreSQL (Neon) + Drizzle ORM
i18n	next-intl 4 (cookie-based, no URL routing)
Charts	recharts 3 (tokenized)
AI	Vercel AI SDK + OpenRouter
Email	Resend (transactional: invites, password reset, proposals)
Billing	Stripe subscriptions (vendor → agency) + webhook
Payments	Stripe Connect (traveler → agency, destination charges)
Flights	Duffel (Amadeus self-service kept only as legacy fallback)
Hotels	Hotelbeds (APITUDE: availability + content)
PDF	<code>@react-pdf/renderer</code> (server-rendered proposals)
Data export	<code>exceljs</code> (XLSX) + built-in CSV (UTF-8 BOM) for BI/Power BI
Storage	Vercel Blob (optional)
Hosting	Vercel + GitHub auto-deploy

Conventions tied to the stack

- **Styling** follows the design system in `DESIGN.md`; see `ui-ux.md`.
- **Working conventions** (sub-agents, UUID ids, Drizzle workflow, testing) live in `AGENTS.md`; see `coding-standards.md`.
- **External integrations** and their env vars are documented in `api-integrations.md`. Every one degrades gracefully when unconfigured.
- **Tailwind v4** is configured CSS-first via `@theme inline` in `globals.css` — there is no `tailwind.config.ts`.

Architecture

Tech choices live in tech-stack.md. This doc covers structure: multi-tenancy, auth, roles, the route map, the module layout, and i18n plumbing.

Multi-tenancy

Every business table carries `agencyId` (tenant roots) or inherits it through a parent (children). **All** reads/writes are scoped by agency, enforced in actions and pages via `requireAgencyUser()` → `user.agencyId`. References (BKG-..., PRD-...) are unique **per agency**. Re-verified by `scripts/test-tenant-isolation.ts`. See `security.md` for the isolation guarantees and `database.md` for tenancy columns.

Auth & onboarding

- **Better Auth** (`src/lib/auth.ts`) — email/password. The `user.create.before` hook makes signup **invitation-only**: it requires a pending `agency_invite` matching the email, stamps `agencyId` + `role`, and rejects everyone else (blocks the raw signup endpoint too).
`BETTER_AUTH_URL/baseURL` + `trustedOrigins` set so the deployed domain is trusted.
- **Guards** (`src/lib/permissions.ts`): `requireUser`, `requireAgencyUser` (tenant + agency-suspension lockout), `requireManager`, `requireCapability`, `requirePlatformAdmin`.

Platform admin (vendor)

A user with `isPlatformAdmin = true`, `agencyId = null` — above all tenants, routed to `/platform`. Cannot enter a tenant app except via impersonation.

Impersonation (cookie-driven, platform-admin only)

- `viewAsAgency(agencyId)` → cookie `platform_view_agency` → acts as agency **admin**.
- `viewAsUser(userId)` → cookie `platform_view_user` (takes precedence) → adopts that user's identity, agency and **role** (full fidelity — an agent sees only their work).
- Resolved in `requireUser`; `user.impersonating = "agency" | "user" | null` drives the exit banner. `exitAgencyView()` clears both cookies.

Per-role landing

`roleHome(role)` routes `finance` → `/finance`, `support` → `/support`, `else` `/dashboard` (which itself adapts: agency-wide for admin/manager, scoped "Your work" for agents). App-shell nav is role-aware via a `show(role)` predicate per item.

Roles & permissions

Defined in `src/lib/domain.ts`. Full capability semantics are in `business-rules.md`.

Role	Sees all data	Team mgmt	Payments	Finance view	Support view	Delete	Home
admin	✓	✓	✓	✓	✓	✓	/dashboard
manager	✓	✓	✓	✓	✓	✓	/dashboard
finance	✓	—	✓	✓	—	—	/finance
support	✓	—	—	—	✓	—	/support
agent	own only	—	—	—	—	—	/dashboard (scoped)

Capability helpers: `seesAllData`, `canManageTeam`, `canAssignAdmin`, `canManagePayments`, `canViewFinance`, `canViewSupport`, `canDeleteRecords`, `roleHome`. Only an **admin** can assign/change the admin role.

Scale targets

The capacity Atlas is designed toward, with implications for the current architecture (single Neon Postgres + Vercel serverless).

Target	Implication / current state
1,000 agencies	Multi-tenant single DB; fine with <code>agencyId</code> indexes (present).
50,000 users	Better Auth + Postgres; fine at this scale.
5,000,000 bookings	Needs solid indexing + pagination (currently hardcoded <code>limits</code> — see tracker).
500,000,000 activities	<code>activity_log</code> at this size needs partitioning + archival/retention — not in place today (single unpartitioned table).
99.9% uptime	Relies on Vercel + Neon SLAs; no DR runbook yet (see <code>security.md</code>).
100 concurrent hotel searches	Bound by Hotelbeds quota; content cache serves photos quota-free. No concurrency/queue control or rate limiting yet.
100 concurrent flight searches	Bound by Duffel quota; <code>safeSearch</code> degrades to sample data on error. No queue/backpressure yet.

These are design targets. The gating work to reach them — table partitioning for `activity_log`, real pagination, rate limiting/backpressure on search, and a DR plan — is in the gap tracker.

Route map

Authenticated app (`(app) /`, gated by `requireAgencyUser`): `dashboard`, `finance`, `commissions`

(canViewFinance), support, bookings (+ new, [id], [id]/edit, [id]/itinerary), clients (+ new, [id], [id]/edit), opportunities (+ new, [id], [id]/edit), products (+ new, [id], [id]/edit), suppliers (+ new, [id], [id]/edit) (canManageTeam), operations (Pipeline board), reports (canViewFinance — BI export hub), search, hotels (+ [code] details), assistant, team (canManageTeam), billing (admin-only), settings, profile.

Client portal (portal/, gated by requirePortalSession): portal (trip list), portal/bookings/[id] (detail + pay), portal/proposals (list), portal/proposals/[id] (view + sign), portal/login.

Platform (platform/, gated by requirePlatformAdmin): platform, platform/agencies/new, platform/agencies/[id].

Auth ((auth)/): login, register (invite-only notice), forgot-password, reset-password. **Accept invite:** invite/[token].

Public / docs: i/[token] (shareable itinerary, unauth), p/[token] (public signable proposal) + p/[token]/pdf, proposal/[id] (internal preview) + proposal/[id]/pdf, booking-docs/[id]/voucher, booking-docs/[id]/invoice.

API: api/auth/[...all] (Better Auth), api/chat (AI assistant), api/export/[entity] (BI export: CSV/XLSX per dataset + workbook = all sheets), api/stripe/webhook (subscription reconciliation), api/stripe/connect-webhook (Connect payment reconciliation), api/portal/auth/request · verify · signout (portal magic-link flow).

Module layout

src/lib/

- domain.ts — roles, capabilities, enums (statuses, stages, item types, currencies + controlled vocabularies: lead source, travel purpose, trip type, gender, title, lost reason, industry), roleHome, status/role metadata.
- analytics.ts — pure dashboard helpers (see analytics.md).
- reference/countries.ts — ISO 3166-1 list (name, demonym, derived flag) + normalizeCountry() alias mapping.
- export/ — csv.ts (RFC-4180 + UTF-8 BOM), xlsx.ts (exceljs workbook), datasets.ts (BI dataset registry with dual code/label columns).
- permissions.ts — auth guards + impersonation resolution.
- auth.ts / auth-client.ts — Better Auth config + client.
- invites.ts — create/find/accept invite tokens (7-day TTL).
- queries.ts — shared agency-scoped pickers.
- activity.ts — logActivity (agency-scoped audit).
- db.ts (validates env via getServerEnv), schema.ts, env.ts, config.ts, format.ts, utils.ts, itinerary.ts, storage.ts.
- notifications/email.ts (Resend adapter) + notifications/templates.ts (HTML).
- billing/stripe.ts — SaaS subscriptions (distinct from payments/stripe.ts).
- payments/stripe.ts — Stripe Connect: account, onboarding link, destination charges.
- suppliers/ — index.ts (getFlightSupplier/getHotelSupplier + safeSearch), duffel.ts, hotelbeds.ts, content-cache.ts, amadeus.ts (legacy), mock.ts, types.ts.

- `documents/proposal-pdf.tsx + proposal-data.ts` — proposal PDF rendering.
- `portal-session.ts` — `getPortalSession / requirePortalSession` (`httpOnly` cookie).

src/lib/actions/ (server actions, all agency-scoped): `clients`, `opportunities`, `products`, `proposals-public`, `bookings`, `payments`, `notifications`, `team`, `invites`, `platform`, `billing`, `settings`, `search`, `suppliers`, `commissions`, `portal-payments`, `portal-proposals`, `portal-invite`, `onboarding`, `ai`.

src/components/ — `app/` (`shell`, `page-header`, `stat-card`, `status-badge`, `getting-started-card`), `charts/` (`insight-charts` incl. `HBarInsight`, `FunnelInsight`), `reference/` (`country-combobox`, `city-input`), `reports/`, `settings/`, `team/`, `platform/`, `auth/`, `bookings/` (`lifecycle stepper`, `search sheet`, `board`), `clients/` (`portal invite`, `timeline`), `products/` (`convert-to-booking`, `AI quote builder`), `opportunities/`, `billing/`, `commissions/`, `suppliers/`, `portal/`, `search/`, `documents/`, `ui/` (`shadcn`).

Internationalization

- **Locales:** `en`, `fr`, `ar` (Arabic = RTL). Cookie-based (`locale`), no URL-locale routing — all routes unchanged.
- `src/i18n/` — `config.ts` (`locales`, `metadata`, `dir`), `request.ts` (next-intl request config reading the `locale` cookie). Messages: `messages/{en,fr,ar}.json`.
- Root layout sets `<html lang dir>` and applies IBM Plex Sans Arabic for RTL (`html[dir="rtl"] body` in `globals.css`).
- Translated surfaces: `nav`, `login`, `dashboard`, `settings`. Others fall back to English. **To translate a string:** add the key to all three `messages/*.json` and swap to `t("...")` (`getTranslations` in server, `useTranslations` in client).
- Language is changed in **Settings**; choice is saved to `user.locale` and the cookie. **Known gap:** a fresh device without the cookie shows English until the user re-picks — `user.locale` isn't yet synced to the cookie on login (see `roadmap.md` open items).

Domain Model

The core business entities and how they relate. This is the **data/relationship** view; the process view (states, who does what) is the golden workflow. Tables and columns live in database.md.

The chain

```
Lead → Client → Opportunity → Proposal → Booking → Supplier →  
Invoice → Payment → Accounting → Travel → Feedback
```

The **client is the aggregate root** — everything hangs off it, and every entity is scoped to an `agencyId` (founding principle #2).

Entities

Domain concept	Table(s)	Status	Key relations
Lead	<code>client</code> (status <code>lead</code> , source code)	✓	becomes a Client; no separate table
Client	<code>client</code> (+ <code>client_contact</code>)	✓	root; owns opportunities, proposals, bookings
Opportunity	<code>opportunity</code>	✓	belongs to a client; <code>assignedToId</code> ; links to a proposal
Proposal	<code>product</code> (+ <code>product_item</code>)	✓	belongs to client; e-sign; converts → Booking
Booking	<code>booking</code> (+ <code>booking_traveller</code> , <code>booking_item</code> , <code>booking_day</code>)	✓	belongs to client; lifecycle states; items reference suppliers
Supplier	<code>supplier</code> (+ <code>supplier_contract</code> , <code>supplier_rate</code>)	✓	referenced by <code>booking_item.supplierId</code> + <code>product_item</code>
Invoice	— (on-demand PDF: <code>booking-docs/[id]/invoice</code>)	●	no managed invoice entity yet (planned module)
Payment	<code>payment</code> (child of booking)	✓	deposits/installments; Stripe Connect
Accounting	<code>commission</code> ledger only	●	partial; no GL/accounting entity (planned module)
Travel	<code>booking</code> (state <code>completed</code>) + <code>booking_day</code>	✓	itinerary timeline; vouchers
Feedback	— (<code>activity_log</code> / notes)	●	no review/feedback entity (planned module)

Aggregates & ownership

- **Client aggregate** — `client` → `client_contact`; the spine that opportunities, proposals, and bookings all reference by `clientId`.
- **Booking aggregate** — `booking` → `booking_traveller`, `booking_item`, `booking_day`, `payment`. Children carry **no** `agencyId`; they inherit tenancy through the parent booking (scoped via the parent on every query).
- **Supplier aggregate** — `supplier` → `supplier_contract` → `supplier_rate`; `commission` rows link a booking item back to the earning supplier/agent.
- **Reference data** — `hotel_content` is **global** (not tenant-scoped); ISO countries/vocabularies are canonical lookups, not per-agency rows.

Where the chain is aspirational

Three links are not yet first-class domain entities — they're the bridge to the planned modules:

- **Invoice** — today an on-demand PDF; the Accounting module turns it into a numbered, ledgered record.
- **Accounting** — only the two-ledger `commission` exists; no general ledger.
- **Feedback** — not modelled; "Travel completed → request review" is an unbuilt automation trigger.

See the entity standard for the baseline fields every one of these should carry.

Database

Schema lives in `src/lib/schema.ts`. PostgreSQL (Neon) + Drizzle ORM. Tenancy column shown where present; see `architecture.md` and `security.md` for how scoping is enforced.

Entity standard

Every business entity should carry this baseline. It operationalizes the never rules (one source of truth, no hard delete, no cross-tenant exposure) at the schema level.

Field	Purpose	Current state
UUID	Primary key, randomly generated (non-Better-Auth IDs)	✓ convention in place
Reference	Human-facing id unique per agency (BKG-..., PRD-...)	⚠ only <code>booking</code> + <code>product</code> today
createdAt	Creation timestamp	✓ widespread
updatedAt	Last-modified timestamp	⚠ on roots, missing on some children
createdBy	<code>createdById</code> → user	✓ on main entities
agencyId	Tenant scope (root) or inherited (child)	✓ enforced everywhere
Status	Lifecycle/state enum	⚠ per-entity, not universal
Activity log	Audit trail of changes	⚠ shared <code>activity_log</code> table, not wired for every entity
Soft delete	<code>deletedAt</code> — never hard delete	✗ no soft-delete column exists yet
Notes	Free-form internal notes	⚠ only some entities

Gap to close: soft delete is **not implemented** — there is no `deletedAt` anywhere, so the "never hard delete" rule is currently aspirational. Adding a nullable `deletedAt` + filtering it out of reads (and a `reference` on the remaining entities) would need a migration. Until then, deletes are real.

Tables

Table	Tenancy	Notes
<code>agency</code>	(root)	name, slug, status (active/suspended); Stripe billing : <code>stripeCustomerId</code> , <code>stripeSubscriptionId</code> , <code>subscriptionStatus</code> , <code>priceId</code> , <code>currentPeriodEnd</code> , <code>trialEndsAt</code> ; Connect : <code>stripeConnectAccountId</code> , <code>stripeConnectOnboarded</code> ; <code>onboardingDismissedAt</code> (getting-started card)

agency_invite	agencyId	email, role, token, status, expiresAt
user	agencyId (nullable)	+ isPlatformAdmin, role, active, locale, commissionRatePercent (Better Auth)
session, account, verification	via user	Better Auth
portal_session	via client	passwordless client portal sessions; token + expiresAt
client	agencyId	+ client_contact (child); source (lead-source code), industry (code), country stored as canonical ISO name
opportunity	agencyId	pipeline stage, value, currency; travel_purpose code; lost_reason code
product	agencyId	proposal; ref unique per agency; + product_item (child; supplierId FK optional); e-sign : shareToken (unique), acceptedAt/declinedAt, signerName/Email, signatureData, signerIp/UserAgent
booking	agencyId	ref unique per agency; shareToken; travel_purpose + trip_type codes
booking_traveller (+ title, gender codes), booking_item (+ supplierId FK), payment, booking_day	via booking	children
notification	agencyId	comms log
activity_log	agencyId	audit trail
supplier	agencyId	managed supplier directory (hotels, airlines, DMC, etc.)
supplier_contract	agencyId	commission basis/rate, validity dates, file URL
supplier_rate	via contract	structured per-product rates within a contract
commission	agencyId	earnings ledger: supplier→agency and agency→agent; bookingId, supplierId, agentUserId, basis, rate, amount, status
hotel_content	(global)	Hotelbeds content cache (photos, facilities, coords) — shared reference data, not tenant-scoped; PK is the Hotelbeds hotel code

ID convention

All ID columns **not** related to Better Auth use UUIDs, randomly generated. See coding-standards.md.

Migrations

Migration	What it adds
0006	Tenancy + backfill (agencyId on all tables)
0007	agency_invite table
0008	user.locale
0009	Agency Stripe billing columns
0010	Product e-signature columns
0011	hotel_content cache
0012	Stripe Connect columns on agency
0013	portal_session table
0014	supplier, supplier_contract, supplier_rate tables; supplierId FK on booking_item + product_item
0015	commission table; user.commissionRatePercent
0016	agency.onboardingDismissedAt
0017	Controlled-vocab columns: client.industry, opportunity.travel_purpose, booking.travel_purpose/trip_type, booking_traveller.title/gender (all nullable, additive)

Migrations 0000–0005 predate the multi-tenant rework (the pre-tenancy base schema) and are not itemized here.

Workflow

```
db:generate → db:migrate # NEVER db:push
```

Run on prod after deploy: `POSTGRES_URL=<prod-url> npx drizzle-kit migrate`. Branches: dev ep-wandering-sunset-aitlty78 · prod ep-misty-thunder-aixz34vy. Full setup in development-guide.md.

API Integrations

Every external integration **degrades gracefully** to sample or logged behaviour when its keys are unset, so the app runs end-to-end with only `POSTGRES_URL` + `BETTER_AUTH_SECRET`. Env vars are summarized in `development-guide.md`.

Provider architecture

Booking providers plug into **one abstraction** so adding Hotelbeds, Amadeus, TravelgateX, Booking.com, or Expedia never touches calling code. Interfaces + registry live in `src/lib/suppliers/providers/` (architecture only — **no provider adapters implemented yet**).

- **Capability-segmented interfaces** (`types.ts`) — a provider implements only what it offers: `HotelSearchCapable`, `HotelBookingCapable`, `FlightSearchCapable`, `FlightBookingCapable`, `CancelCapable`. A hotels-only aggregator implements the hotel interfaces and nothing else (no empty stubs).
- **Full booking lifecycle** — `search` → `quote` (re-validate price/rate) → `book` (idempotent) → `cancel`. Real bedbank/GDS rates expire, so `quoteHotel/quoteFlight` re-price immediately before `book*`; `book` requests carry an `idempotencyKey` so replays never double-book.
- **Normalized errors** — every provider throws `ProviderError` with a stable code (`rate_expired`, `sold_out`, `rate_limited`, `provider_unavailable`, ...) and a `retryable` flag, so callers branch on the code, not provider strings.
- **Tenant-aware context** — every call takes a `ProviderContext` (`agencyId`, `currency`, `locale`, `correlationId`, abort signal); providers never read globals.
- **Registry** (`registry.ts`) — adapters `register()` themselves at startup; callers resolve by vertical + capability + priority (`providerRegistry.pick("hotels", "search")`) with capability type-guards (`canSearchHotels`, `canBookFlights`, ...). Replaces the hardcoded `getFlightSupplier()/getHotelSupplier()` if/else.
- **Provider catalog** (`PROVIDER_CATALOG`) — one logic-free source of truth for which providers exist, their verticals/capabilities, env vars, and status (`live` / `legacy` / `planned`):

Provider	Verticals	Status
Mock	flights + hotels	live
Duffel	flights	live
Amadeus	flights (+ hotels)	legacy
Hotelbeds	hotels	live
TravelgateX	hotels + flights	planned
Booking.com	hotels	planned
Expedia (Rapid)	hotels	planned

Migration path: the current `SupplierProvider` (search-only, both-verticals) keeps working; new adapters implement these interfaces and register into the registry, then `getFlightSupplier/`
`getHotelSupplier` delegate to `providerRegistry.pick(...)`. Real booking (Open item #1) lands
provider-by- provider against `quote/book/cancel` without further interface changes.

Flights — Duffel

- `src/lib/suppliers/duffel.ts`, behind `getFlightSupplier()` + `safeSearch()`.
- Airport autocomplete, one-way/round-trip, flight codes, connecting airports.
- Falls back to sample data when `DUFFEL_API_TOKEN` is unset.
- Amadeus self-service (`amadeus.ts`) is kept **only as a legacy fallback**.
- **Search-only today** — placing real orders is an open item (roadmap.md).

Hotels — Hotelbeds (APITUDE)

- `src/lib/suppliers/hotelbeds.ts` + `content-cache.ts`.
- Availability + content APIs. Booking.com-style search/results/details, dynamic occupancy pricing, filters, room photos, compare.
- **Content cache** (`hotel_content` table, global/non-tenant) serves real photos, facilities and coords quota-free; synced via `scripts/sync-hotel-content.ts`.
- **Search by hotel name** resolves matching hotels via the Content API (`searchHotelbedsHotelsByName`) then prices them live; falls back to estimated rates.
- Keys: `HOTELBEDS_API_KEY` / `HOTELBEDS_SECRET`. Falls back to sample data when unset.
Search-only today (real booking is an open item).

Supplier abstraction

`src/lib/suppliers/index.ts` exposes per-vertical `getFlightSupplier / getHotelSupplier` and a `safeSearch` wrapper that catches provider errors and returns mock results, so the UI never hard-fails. `mock.ts` provides the sample data; `types.ts` the shared shapes. This is separate from the managed **supplier directory** (the `supplier` table — agency's own hotel/airline/DMC contacts; see `business-rules.md`).

Billing — Stripe subscriptions (vendor → agency)

- `src/lib/billing/stripe.ts`. Vendor bills agencies; 14-day trial on provision.
- Checkout, billing portal, and **manual webhook signature verification**.
- `api/stripe/webhook` reconciles subscription status; `requireAgencyUser` gates on a lapsed subscription.
- Keys: `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`.

Payments — Stripe Connect (traveler → agency)

- `src/lib/payments/stripe.ts` — **distinct** from `billing/stripe.ts`.

- Agencies onboard a connected Express account to receive traveler payments directly (**destination charges**); the platform takes a configurable fee.
- `api/stripe/connect-webhook` reconciles Connect payments. Powers the client portal "Pay now".
- Keys: `STRIPE_CONNECT_WEBHOOK_SECRET`, `STRIPE_PLATFORM_FEE_PERCENT`.

Email — Resend

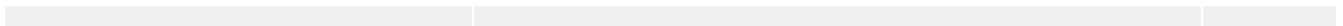
- `src/lib/notifications/email.ts` (adapter) + `templates.ts` (HTML).
- Sends invite emails, password-reset, proposal acceptance, portal invites.
- When unconfigured, logs to console + the `notification` table; invite/portal links are made copyable so the flow still works.
- Keys: `RESEND_API_KEY`, `EMAIL_FROM`.

AI — Vercel AI SDK + OpenRouter

- Powers the assistant and inline AI features; see `ai.md`.
- Key: `OPENROUTER_API_KEY`. AI features no-op/degrade when unset.

Storage — Vercel Blob (optional)

- Supplier contract PDF uploads. Key: `BLOB_READ_WRITE_TOKEN`.



Development Guide

Setup

```
npm install --legacy-peer-deps      # better-auth peer range
npm run dev                          # http://localhost:3000
npm run check                        # lint + typecheck
npm run build:ci                     # next build
```

After completing any change, run `npm run check` and `npm run build:ci` to verify code quality (lint, typecheck, build). See `coding-standards.md`.

Environment (`.env`)

Required: `POSTGRES_URL`, `BETTER_AUTH_SECRET`.

Optional (all integrations degrade gracefully to sample/logged behaviour when unset — see `api-integrations.md`):

- `OPENROUTER_API_KEY` — AI features
- `BLOB_READ_WRITE_TOKEN` — uploads
- `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET`
- `NEXT_PUBLIC_APP_URL`
- `RESEND_API_KEY` / `EMAIL_FROM` — email
- `STRIPE_SECRET_KEY` / `STRIPE_PRICE_ID` / `STRIPE_WEBHOOK_SECRET` — billing
- `STRIPE_CONNECT_WEBHOOK_SECRET` / `STRIPE_PLATFORM_FEE_PERCENT` — Connect payments
- `DUFFEL_API_TOKEN` — flights
- `HOTELBEDS_API_KEY` / `HOTELBEDS_SECRET` — hotels
- `PROTECTED_DB_HOSTS` — makes destructive scripts refuse a prod DB (override with `ALLOW_PROD=1`). See `security.md`.

Database

- Branches: `dev` `ep-wandering-sunset-aitlty78` · `prod` `ep-misty-thunder-aixz34vy`.
- After schema changes: `db:generate` → `db:migrate` (**never** `db:push`). `db:studio` to browse. See `database.md`.
- **Always run migrations on prod after deploy:** `POSTGRES_URL=<prod-url> npx drizzle-kit migrate`.

Maintenance scripts

```
# Promote an existing account to the platform super-admin
npx tsx --env-file=.env scripts/make-platform-admin.ts <email>
```

```
# Seed / reset the Demo Agency (idempotent — wipes its data, keeps users, reseeds)
npx tsx --env-file=.env scripts/seed-demo-data.ts

# Cross-tenant isolation test (seeds 2 agencies, asserts no leak, cleans up)
npx tsx --env-file=.env scripts/test-tenant-isolation.ts

# Normalize legacy free-text country values to canonical ISO names
npx tsx --env-file=.env scripts/backfill-countries.ts

# Sync Hotelbeds hotel content (photos/facilities/coords) into the cache table
npx tsx --env-file=.env scripts/sync-hotel-content.ts # curated destinations
npx tsx --env-file=.env scripts/sync-hotel-content.ts BCN MAD # specific codes
```

Testing

Use the project's testing tools/scripts (e.g. `test-tenant-isolation.ts`) to verify changes — never assume changes work. If no tooling covers a change, ask whether testing should be skipped. See [coding-standards.md](#).

Coding Standards

Source of truth: `AGENTS.md` (aliased by `CLAUDE.md`) holds the enforced working conventions. This doc summarizes and points there — it does not restate or override it.

Working conventions (from AGENTS.md)

- **Responses** — keep concise and to the point unless asked otherwise.
- **Planning** — always ask clarifying questions; never assume design, stack or features; use deep-dive sub-agents for research and to review plans.
- **Change/edit mode** — prefer sub-agents to implement features; act as a coordinator; parallelize independent work; use premium models for complex tasks (coding) and mid-tier for simpler ones (documentation).
- **Quality gates** — after completing features, run lint, type check and `next build` (`npm run check`, `npm run build:ci`). See `development-guide.md`.
- **Testing** — use available testing tools; never assume changes work; if no tooling exists for a change, ask whether to skip.

Engineering rules

Hard rules for how code is written. Status reflects the current codebase (✓ holds · ● partial).

Rule	Status	Notes
Never duplicate logic	●	Shared helpers in <code>src/lib</code> (<code>queries</code> , <code>analytics</code> , <code>domain</code>); enforce by review.
Always use server actions	✓	20/21 files in <code>src/lib/actions</code> are <code>"use server"</code> ; mutations go through them.
Always validate input	●	Validated where it matters, not universally.
Always use Zod	●	Zod v4 present, used in ~13 files; not every action yet.
No business logic inside components	●	Logic lives in <code>actions/lib</code> ; enforce by review.
No direct SQL in UI	✓	Pages call Drizzle queries server-side / via actions, not raw SQL in client components.
Every action logs activity	●	<code>logActivity</code> in ~11/21 action files — not yet every mutation (see <code>security.md</code>).
Every mutation is tenant scoped	✓	All actions go through <code>requireAgencyUser()</code> → <code>agencyId</code> scope (see <code>business-rules.md</code>).

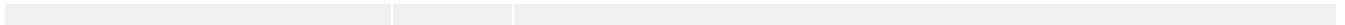
The ● rows (universal Zod validation, universal activity logging) are in the spec-vs-reality gap tracker.

Database changes

- After any schema change, run **drizzle generate + migrate**. **Never** `db:push`. Workflow detail in `database.md`.
- All ID columns **not** related to Better Auth use **UUIDs**, randomly generated.

UI

- Follow the design system in `DESIGN.md` when creating or reviewing components/pages. See `ui-ux.md`.



UI / UX

Source of truth: `DESIGN.md` defines the full visual design system — colors (oklch tokens), typography, spacing, radius, shadows, animations, layout patterns, shadcn/ui component conventions, and dark mode. All new components and pages **must** follow it. This doc points there and captures product-level UX patterns only; it does not duplicate the token tables.

Atlas design principles

The UI-level expression of the ten founding Atlas principles. The product guardrails every page and feature is measured against:

1. **Never ask the user twice for the same information.** Reuse what's already captured (controlled vocabularies, reference data, client records) instead of re-prompting.
2. **Everything starts from the client.** The client record is the spine — leads, opportunities, proposals, bookings and payments all hang off it.
3. **Every action must be reversible.** Prefer soft states, confirmations and undo over destructive, irreversible operations.
4. **One source of truth.** No duplicated or conflicting data; canonical values (ISO countries, enums, references unique per agency) over free text.
5. **Automation before manual work.** Default to generating (commissions, itineraries, quotes, documents) rather than asking the user to do it by hand.
6. **Every page should answer "What should I do next?"** Surface the next action — stepper advance, CTA, empty-state prompt — never leave a dead end.
7. **Managers want insights; agents want speed.** Tailor each role's surface — analytics-dense for managers, fast and scoped for agents.
8. **No empty pages.** Every list/chart/detail has an empty state with a clear next step, never blank space.
9. **Data quality is more important than flexibility.** Constrain inputs (dropdowns, pickers, validation) to keep reporting clean.
10. **Everything should be mobile friendly.** Layouts work on small screens and in both LTR and RTL.

These principles inform the patterns below and the business rules. See also the product vision.

Design system summary

- **Stack:** Next.js + Tailwind v4 (CSS-first `@theme inline`, no config file), shadcn/ui (new-york, neutral), Lucide icons, Geist fonts, next-themes dark mode.
- **Tokens:** semantic oklch color tokens, `--radius 10px` base, Tailwind-default shadows, custom `fade-in/fade-up/scale-in` animations. Use `cn()` for class merging.
- See `DESIGN.md` for every value.

Performance budgets

Target response times — what "fast enough" means per surface. These back design principle #6 (agents want speed) and dictate where to lean on caching, streaming, and loading states.

Surface	Budget
Dashboard	< 1 s
Search (in-app lookup)	< 500 ms
Booking creation	< 2 s
Hotel search (live Hotelbeds)	< 5 s
Reports / export	< 10 s

Budgets are **targets, not yet measured** — there's no perf instrumentation in place. The slow surfaces (hotel search, reports) are I/O-bound on external APIs and the export workbook; the content cache already keeps hotel photos quota- and latency-free (api-integrations.md). Every surface over its budget must show a loading state.

Data table standard

The target capabilities for every data table (clients, bookings, opportunities, proposals, suppliers, commissions). These belong in **one reusable DataTable component**, not re-implemented per page:

- **Sorting** — click a column header to sort; multi-column where useful.
- **Filtering** — per-column filters plus the page-level search/filter bar.
- **Column chooser** — show/hide columns; remember the user's choice.
- **Export** — export the current view (respecting sort/filter) to CSV/Excel via the existing `/api/export` endpoint.
- **Infinite scrolling** — load more on scroll instead of hard `limit` truncation (fixes the silent-cap issue in the page audit).
- **Sticky header** — header stays visible while the body scrolls.
- **Keyboard shortcuts** — arrow-key row navigation, Enter to open, / to focus search.
- **Context menu** — right-click a row for quick actions (open, edit, duplicate, delete — gated by permissions).

Current state: tables today are plain `shadcn Table` markup with none of the above — no sorting, column chooser, infinite scroll, sticky header, shortcuts or context menu, and a hardcoded row `limit`. This standard is the target for a shared `DataTable`; building it also clears most of the systemic gaps in the page audit (export, pagination, bulk actions).

Page requirements checklist

Every list/index page must ship with all of these — a page missing any of them is incomplete, not "v1":

- [] **Search** — free-text lookup over the page's records
- [] **Filters** — by the page's key dimensions (status, type, date range, owner)
- [] **Export** — CSV/Excel of the current view (see analytics.md)
- [] **Bulk actions** — multi-select + an action that applies to the selection
- [] **Empty state** — a prompt with a clear next step (never blank — principle #8)
- [] **Loading state** — skeleton/spinner while data resolves
- [] **Error state** — a recoverable message, not a crash
- [] **Pagination** — bounded result sets, never an unbounded dump
- [] **Mobile layout** — works on small screens, LTR and RTL (principle #10)
- [] **Permissions** — role-scoped data + role-gated actions (see business-rules.md)

This operationalizes the design principles at the page level. Detail pages additionally answer "what should I do next?" via a primary CTA or lifecycle action.

Product UX patterns

These are app-specific patterns layered on the design system:

- **Getting-started checklist** — dismissible 4-step card on the dashboard for new agencies; dismissed state persists in DB (`agency.onboardingDismissedAt`) across devices and team members.
- **Lifecycle stepper** — horizontal progress bar on booking detail with an "Advance to [next]" button and soft/hard prerequisite guards (see business-rules.md).
- **Role-gated nav** — locked nav items shown dimmed with a tooltip for non-admin roles; nav is filtered by a `show(role)` predicate.
- **List/Board toggle** — Bookings and Pipeline share data with a table↔kanban switch.
- **Inline search sheet** — "Search flights/hotels" on the trip-services panel opens a sheet pre-scoped to the booking's destination.
- **Consolidated sharing** — proposal sharing lives under one "Share with client ▼" dropdown.
- **Empty states** — charts and lists show empty-state badges/CTAs rather than blank space.

RTL & i18n

Arabic is RTL (IBM Plex Sans Arabic, `html[dir="rtl"]`). Layout must work in both directions. i18n plumbing is in architecture.md.

Business Rules

Domain logic and constraints. Enums and capability helpers are defined in `src/lib/domain.ts`; server-side enforcement lives in the actions under `src/lib/actions/`.

Automation triggers

The event → action automation layer Atlas is targeting (design principle #5, *automation before manual work*). Each is a domain event that should fire a follow-up action:

Event	Action	Current state
Client created	Send welcome email	✗ not implemented (Resend only sends invites/reset/proposal/portal)
Opportunity won	Generate proposal	✗ manual (new proposal + AI quote builder)
Proposal accepted	Generate booking	⚠ one-click <i>convert proposal</i> → <i>booking</i> , not automatic
Booking confirmed	Generate invoice	✗ invoices are on-demand PDFs, not auto-generated
Payment received / booking confirmed	Update accounting	⚠ partial — <code>autoGenerateCommissions</code> fires on confirm/ticket (bookings.ts); no full accounting
Travel completed	Request review	✗ not implemented

Current state: the only true event-driven automation today is commission generation on booking confirm/ticket (idempotent). The rest are manual or on-demand. Building this trigger layer is the heart of the "Automation" pillar in the vision and the automated-quotation item in the roadmap.

Never rules (hard constraints)

Invariants Atlas must never violate. Each maps to an enforcement point — a violation is a bug, not a preference.

Never	Why / where enforced
Never use free text when a dropdown exists	Controlled vocabularies + reference pickers keep reporting clean (data quality > flexibility). See vocabularies.
Never duplicate client information	One source of truth — the client record is the spine; reuse it, don't re-enter it.
Never create a booking without a client	Every booking hangs off a client; the spine is mandatory.

Never hard delete data	Reversibility — prefer soft state / archival + the activity log; deletes are gated to admin/manager only.
Never expose another tenant	Every read/write is scoped by <code>agencyId</code> via <code>requireAgencyUser()</code> ; verified by <code>test-tenant-isolation.ts</code> . See <code>security.md</code> .
Never sum different currencies	No FX — group by currency with a DZD headline. See <code>Currency</code> and <code>analytics.md</code> .
Never make users navigate five pages	Surface work where it happens — inline search sheets, detail-page panels, "what next?" CTAs.
Never require more than three clicks	Key actions (advance lifecycle, convert proposal→booking, share, pay) are one-to-three clicks.

These are the enforcement edge of the design principles.

Golden workflow

The canonical end-to-end happy path Atlas is built around. Everything starts from the client (design principle #2) and ends by feeding the next trip — a loop, not a line.

```
Lead → Client → Opportunity → Proposal → Customer accepts → Booking →
Supplier reservation → Payment → Ticketing → Travel → Feedback → Repeat client
```

Step	In Atlas	Notes
Lead	<code>client</code> with a <code>source</code> (lead-source code)	captured as an early-stage client
Client	<code>client</code> (+ contacts)	the spine; all records hang off it
Opportunity	<code>opportunity</code> (pipeline stage, value, <code>travel_purpose</code>)	shown on the Pipeline board
Proposal	<code>product</code> (PDF + e-sign)	shareable <code>/p/[token]</code> or in-portal
Customer accepts	e-sign stamps signer + flips opportunity to won	also acceptable in the client portal
Booking	<code>booking</code> , one-click convert proposal → booking	lifecycle starts at <code>draft</code>
Supplier reservation	<code>booking_item</code> + <code>supplier</code> picker	live Duffel/Hotelbeds search; real reservation is an open item (roadmap.md)
Payment	<code>payment</code> (deposit/installments, Stripe Connect)	<code>confirmed</code> requires zero balance
Ticketing	<code>lifecycle</code> <code>ticketed</code>	requires trip items; auto-generates commissions
Travel	<code>lifecycle</code> <code>completed</code> ; <code>itinerary</code> <code>/i/[token]</code>	day-by-day timeline, vouchers/invoices

Feedback	activity log / notes on the client timeline	closes the loop
Repeat client	back to Opportunity on the same client	retention, not re-acquisition

This maps onto the booking lifecycle below and the role landings in architecture.md.

Roles & capabilities

Five roles: **admin**, **manager**, **finance**, **support**, **agent**. The capability matrix is in architecture.md. Key rules:

- `seesAllData` — all roles except **agent** (agents see only their own records).
- `canManageTeam`, `canManagePayments` — admin/manager.
- `canViewFinance` — admin/manager/finance (gates `/finance`, `/commissions`, `/reports`).
- `canViewSupport` — admin/manager/support.
- `canDeleteRecords` — admin/manager.
- `canAssignAdmin` — **only an admin** can assign or change the admin role.

Onboarding & access

- **Invitation-only signup** — enforced at the auth layer (`user.create.before` hook). No invite ⇒ no account. See security.md.
- **Agency suspension** — a suspended agency locks out its users via `requireAgencyUser`.
- **Lapsed subscription** — gates access via `requireAgencyUser` (14-day trial on provision). See api-integrations.md.

Currency

- Supported: **DZD (default)**, EUR, USD.
- **No FX conversion**. The agency operates in DZD; Atlas never sums across currencies. Analytics group **by currency** with a DZD headline. See analytics.md.

References

- BKG-... (bookings) and PRD-... (proposals) reference numbers are unique **per agency**, not globally.

Booking lifecycle

States: `draft` → `awaiting_payment` → `confirmed` → `ticketed` → `completed`. A visual stepper drives it with an "Advance to [next]" button. **Hard prerequisites are enforced server-side:**

- `confirmed` requires trip items **and** zero outstanding balance.

- `ticketed` requires trip items.
- Vouchers/invoices are **blocked** when a booking has no trip services.

Proposals & e-signature

- Server-rendered PDF; shareable via public tokenized `/p/[token]` (no login) and in-portal signing.
- E-sign stamps signer name/email/IP/UA and flips the linked opportunity to **won**.
- **Convert accepted proposal** → **booking** in one click. (A `convertedBookingId` guard column is a pending open item — see roadmap.md.)

Commissions

- **Two ledgers:** `supplier_to_agency` (agency earns from a supplier per booking item) and `agency_to_agent` (agent earns from the agency per booking).
- **Auto-generated** when a booking is confirmed or ticketed; **idempotent**.
- Agent rate from `user.commissionRatePercent`; supplier basis from the supplier contract (percent / fixed / net).

Controlled vocabularies

Seven fields are enum dropdowns (codes stored, labels shown) for clean reporting: lead source, travel purpose, trip type, gender, title, lost reason, industry. Country/nationality use canonical ISO 3166-1 values (full name stored) so spellings never drift; city uses curated autocomplete suggestions.

Client portal

- Magic-link passwordless sessions scoped to **one client**, separate from staff auth (`portal_session`).
 - Agents send the invite from a client or booking detail page; the link is copyable when email is unconfigured.
 - Online "Pay now" appears only when the agency has onboarded Stripe Connect.
 - Clients can accept/decline proposals in-portal with the same e-sign audit as the public flow.
-

Deployment

Vercel

- Project `atlasproject/agence_tool`, connected to GitHub.
- **Auto-deploy:** `git push origin main` → builds & ships production.
- **Manual:** `npx vercel deploy --prod --yes`.
- `vercel.json` → `buildCommand: npm run build:ci; .npmrc → legacy-peer-deps=true`.

Vercel environment

`POSTGRES_URL`, `BETTER_AUTH_SECRET`, `NEXT_PUBLIC_APP_URL`, `BETTER_AUTH_URL`,
`PROTECTED_DB_HOSTS=ep-misty-thunder-aixz34vy` (plus integration keys as needed — see `api-integrations.md`).

Migrations on prod

Always run migrations on prod after a deploy that changed the schema:

```
POSTGRES_URL=<prod-url> npx drizzle-kit migrate
```

Branches: `dev` `ep-dawn-voice-ai8d6q3o` · `prod` `ep-misty-thunder-aixz34vy`.

`PROTECTED_DB_HOSTS` makes destructive scripts refuse the prod DB unless `ALLOW_PROD=1`. See `security.md` and `database.md`.

Demo accounts

Throwaway accounts on the **Demo Agency**, all on the live site.

Role	Email	Password
Platform admin (vendor)	<code>ouksili.abdelmalek@gmail.com</code>	<code>Atlas!2026</code>
Manager	<code>yasmine@agence.test</code>	<code>Agency!2026</code>
Finance	<code>finance@demo.test</code>	<code>Finance!2026</code>
Support	<code>support@demo.test</code>	<code>Support!2026</code>
Agent	<code>karim@demo.test</code>	<code>Agent!2026</code>
Agent	<code>lina@demo.test</code>	<code>Agent!2026</code>
Agent	<code>omar@demo.test</code>	<code>Agent!2026</code>

 **DEMO CREDENTIALS ONLY** — rotate or delete before any real production use. These live

passwords are checked into the repo; treat as a known risk (see security.md).

Suggested demo flow: sign in as vendor → /platform → View as Demo Agency → manager dashboard → switch to finance/support/agent views → Settings → switch to Français or العربية.

Roadmap & Changelog

Phase status

Phase 1 — Sellable ✓ COMPLETE Email (Resend) · Stripe SaaS billing · Separate Neon DB branches (dev/prod) · PDF proposals · E-signature · Live flights (Duffel) · Live hotels (Hotelbeds + content cache).

Phase 2 — Competitive ✓ COMPLETE Supplier management (/suppliers, contracts, rates, picker) · Commissions tracking (auto-generated, two-ledger, /commissions) · Client portal (magic-link login, trip view, online payments via Stripe Connect, in-portal proposal signing + portal invite from agent).

Phase 3 — AI differentiation ✓ COMPLETE AI itinerary generation · AI quote builder · AI email drafting · AI visa assistant — all inline, embedded where the work happens. See ai.md.

UX overhaul ✓ COMPLETE (20 changes across 4 size categories) Getting-started checklist · Lifecycle stepper · Board view toggle · Inline search · Portal invite · Convert proposal→booking · Client funnel timeline · Booking hard guards · Vocabulary pass · Role nav tooltips · Empty-state badges.

Data quality & BI ✓ COMPLETE (4 phases) DZD-only currency (no FX) · **P1** richer analytics (revenue/conversion/pipeline funnel/AR aging/margin) · **P2** controlled-vocabulary enums (7 fields, codes+labels) · **P3** ISO country/nationality reference data + city suggestions · **P4** standardized CSV/Excel export (/reports, dual code+label columns). Hotel search-by-name added. See analytics.md.

Module maturity

Maturity levels: **Production** (stable, in daily use) · **Beta** (works, hardening) · **Alpha** (functional, evolving) · **Planned** (not built).

Module	Status
CRM	Production
Sales	Production
Hotels	Beta
Flights	Planned
Accounting	Planned
AI	Alpha
Supplier Portal	Planned

Flights/Hotels nuance: live *search* is shipped for both (Duffel + Hotelbeds); the maturity above reflects the end-to-end **booking** flow — Hotels is further along (Beta), real Flights booking is Planned (Duffel orders). See api-integrations.md and Open item #1.

Open items

1. **Real supplier booking** — Duffel orders + Hotelbeds book API (currently search-only).
2. **Translate deeper pages** — bookings, clients, finance, support, platform (i18n plumbing ready).
3. **Cross-device locale** — sync `user.locale` → cookie on login.
4. **WhatsApp integration** — Meta Cloud API adapter (skeleton ready; Meta Business account needed).
5. **Automated quotations** — trigger quote generation from opportunity stage change.
6. **Agent performance scoring** — leaderboard from commission + booking counts.
7. **Convert proposal to booking guard** — add `convertedBookingId` column (needs migration).

Planned modules

The longer-horizon module backlog, grouped. Some already have partial foundations (noted) — the rest are net-new.

Accounting

- Accounting · General Ledger · Payroll — net-new finance suite.
- Invoices —  on-demand invoice PDFs exist today; this is the move to managed, numbered, ledgered invoices.

Marketing

- Marketing · Email campaigns — net-new (transactional email via Resend exists; campaigns do not).
- WhatsApp —  adapter skeleton ready (open item #4); needs a Meta Business account.
- SMS — net-new channel adapter.

Documents

- Document management — net-new (today: PDF generation + Vercel Blob uploads for supplier contracts).

Travel products

- eVisa · Visa tracking —  an AI visa assistant exists; these add structured application + tracking.
- Insurance —  exists as a trip-service item type; this is a managed insurance product/flow.
- Vehicle rental · Cruises — net-new verticals (alongside flights/hotels).
- Package builder — net-new (bundle multi-service trips).

Ecosystem

- Supplier portal — net-new (suppliers self-serve, parallel to the client portal).
- Customer loyalty · Affiliate program — net-new growth modules.
- API marketplace — net-new (expose/consume third-party integrations).

These are directional, not scheduled. Phase status and the active gap list are above; this is the

"what's next after the core is hardened" backlog.

Spec vs. reality gap tracker

The design system (principles, never-rules, page/table/entity standards, AI guardrails, automation triggers, perf budgets) describes a **target state**. This table consolidates every item where the spec is ahead of the code, so the gaps are one punch list instead of scattered "current state" notes.

Status: ✓ done · ● partial · ● not started

Item	Spec	Status	Notes
Agent visibility scoped to own records on list pages	security	✓	Fixed — clients/bookings/opportunities/products now scope by owner column (commit 134ceb3).
Quality gate <code>npm run check</code> works (pnpm v11)	development-guide	✓	Fixed — config moved to <code>pnpm-workspace.yaml</code> ; 0 lint/type errors (commit 7d5f4e1).
Soft delete (<code>deletedAt</code> , filtered reads)	database	●	No soft-delete column anywhere; "never hard delete" is currently aspirational. Needs migration.
<code>reference</code> on all entities	database	●	Only <code>booking + product</code> ; missing on clients/opportunities/suppliers. Needs migration.
Consistent <code>updatedAt/status/notes</code> on children	database	●	Present on roots, inconsistent on child tables.
Page systemic-five: per-page Export	ui-ux	●	Endpoint exists (<code>/api/export</code>); list pages lack buttons.
Page systemic-five: Bulk actions	ui-ux	●	No <code>Checkbox</code> primitive exists; nothing built.
Page systemic-five: Loading states	ui-ux	●	Only <code>assistant/dashboard</code> have <code>loading.tsx</code> . <code>skeleton.tsx</code> exists, unused.
Page systemic-five: Error states	ui-ux	●	Only <code>assistant</code> has <code>error.tsx</code> .
Page systemic-five: Pagination	ui-ux	●	Hardcoded <code>limit</code> (200/500) silently truncates.
Shared DataTable (sort, filter, column chooser, infinite scroll, sticky header, shortcuts, context menu)	ui-ux	●	Tables are plain <code>shadcn</code> markup. Building this clears most of the systemic-five.
Mobile-friendly tables (<code>overflow-x-auto</code>)	ui-ux	●	Tables overflow on phones; only <code>assistant</code> wraps for scroll.

Automation triggers (welcome email, won→proposal, accepted→booking, confirmed→invoice, completed→review)	business-rules	●	Only commission auto-gen on confirm/ticket fires; rest manual/on-demand.
AI mutation behind a hard confirm step	ai	●	<code>createBooking</code> is intent-gated by prompt only, not an enforced confirm gate.
Performance instrumentation vs budgets	ui-ux	●	Budgets defined; nothing measured.
Real supplier booking (Duffel orders, Hotelbeds book)	api-integrations	●	Search-only today (also open item #1 above).
Cross-device locale sync	architecture	●	English until re-pick on a fresh device (also open item #3 above).
Rate limiting (auth + API)	security	●	No throttling anywhere.
GDPR subject export/erasure + consent	security	●	No flow; erasure also blocked by missing soft delete.
Disaster recovery runbook + RTO/RPO	security	●	Neon has provider backups; no documented DR plan.
Universal audit-log coverage	security · coding-standards	●	<code>logActivity</code> in ~11/21 action files; not all mutations logged.
Universal Zod input validation	coding-standards	●	Zod in ~13/21 action files; not every action validates.
<code>activity_log</code> partitioning + retention (500M-row target)	architecture	●	Single unpartitioned table; no archival.
Search concurrency/backpressure (100 concurrent)	architecture	●	No queue or rate control on hotel/flight search.

Changelog

Commit	Summary
a6def26	Phase 4: standardized BI data export (CSV + Excel) at <code>/reports</code>
148d1fb	Phase 3: country/nationality reference data + city suggestions
e09289f	Phase 2: controlled vocabularies (enums) for cleaner reporting
0f6840f	Phase 1 analytics: revenue/conversion/pipeline insights (DZD)
bf71024	Enrich demo seed: 100 clients, suppliers, commissions, notifications (DZD)
0b5de21	Restrict currencies to DZD/EUR/USD, default DZD; add hotel name search
441fd6e	UX large: persistent onboarding (DB), client timeline, booking hard guards

284d00e	UX medium: lifecycle stepper, board view, inline search, getting-started card
805dbc8	UX small: portal invite, convert proposal→booking, client funnel view, share consolidation
9c7cc98	UX quick wins: vocabulary, nav labels, empty charts, dashboard CTA
23ddc35	Phase 2 & 3: supplier mgmt, commissions, client portal, AI features
94aaf08	Phase 2: real bookings, Stripe Connect, traveler portal, full i18n
2dd7650	Embed live flight/hotel search in booking Add dialog
a677b77	Fix duplicate-reference crash on create booking/proposal
169ee74	Split atlas.md into focused docs under docs/atlas/
8c1b24e	Hotel module: full search/results/details, dynamic occupancy pricing, content cache
b67cfa0	Phase 1: email (Resend), Stripe billing, e-sign, PDF, suppliers
a233d32	View-as-user + i18n (EN/FR/AR + RTL) + Settings hub
1896596	Per-role workspaces (Finance + Support) + role-based landing/nav
9e8fb4b	Multi-tenant architecture + vendor platform console

Migrations: 18 (latest 0018). Prod DB: ep-misty-thunder-aixz34vy.

Security

Security controls

The control areas Atlas is held to, with honest current status (✓ in place · ● partial / provider-level · ○ not implemented). Details for the implemented ones follow in the sections below.

Control	Status	Notes
Tenant isolation	✓	<code>agencyId</code> scoping via <code>requireAgencyUser()</code> ; verified by <code>test-tenant-isolation.ts</code> .
RBAC	✓	5 roles + capability helpers; agent visibility now scoped on list pages too.
Audit logs	●	<code>activity_log</code> + <code>logActivity</code> used in ~11 action files; not yet universal.
Encryption	●	TLS in transit + Neon at-rest (provider); Better Auth hashes passwords. No app-level field encryption.
Password policy	●	Better Auth defaults only; no enforced strength/rotation policy.
Secrets management	●	Vercel env vars; demo credentials committed in the repo (rotate before prod). No vault.
Backups	●	Neon automatic backups + branching (provider-level); not app-managed or restore-tested.
Rate limiting	●	None on auth or API routes.
GDPR	●	No subject data-export/erasure flow or consent tracking; right-to-erasure also blocked by missing soft delete.
Disaster recovery	●	No documented DR runbook or RTO/RPO targets.

The ●/● rows are tracked in the spec-vs-reality gap tracker.

Tenant isolation

Every business table carries `agencyId` (root) or inherits it via a parent (children). **All** reads/writes are scoped by agency through `requireAgencyUser()` → `user.agencyId`. Reference numbers (BKG-..., PRD-...) are unique per agency. Cross-tenant leakage is re-verified by `scripts/test-tenant-isolation.ts` (seeds two agencies, asserts no leak, cleans up). See `architecture.md`.

Authentication

- Better Auth (email/password). Signup is **invitation-only**: the `user.create.before` hook requires a matching pending `agency_invite`, stamps `agencyId` + `role`, and rejects everyone else — this also blocks the raw signup endpoint.

- `BETTER_AUTH_URL / baseURL + trustedOrigins` restrict trusted origins to the deployed domain.
- Invite tokens have a 7-day TTL (`src/lib/invites.ts`).

Authorization guards

`src/lib/permissions.ts`: `requireUser`, `requireAgencyUser` (tenant + agency-suspension + lapsed-subscription logout), `requireManager`, `requireCapability`, `requirePlatformAdmin`. Capability rules in `business-rules.md`. Only an **admin** can assign/change the admin role.

Platform admin & impersonation

- Platform admin: `isPlatformAdmin = true`, `agencyId = null`; sits above all tenants and **cannot enter a tenant except via impersonation**.
- Impersonation is **cookie-driven and platform-admin only** (`platform_view_agency / platform_view_user`), resolved in `requireUser`, with a persistent exit banner. See `architecture.md`.

Client portal sessions

Passwordless magic-link sessions (`portal_session`) are scoped to a single client and **separate** from staff auth, stored in an `httpOnly` cookie (`portal-session.ts`).

Webhooks

Stripe webhooks use **manual signature verification** before acting: `api/stripe/webhook` (subscriptions) and `api/stripe/connect-webhook` (Connect payments). Secrets: `STRIPE_WEBHOOK_SECRET`, `STRIPE_CONNECT_WEBHOOK_SECRET`.

Database safety

`PROTECTED_DB_HOSTS` makes destructive scripts **refuse to run against a protected (prod) host** unless explicitly overridden with `ALLOW_PROD=1`. In production it is set to the prod branch host `ep-misty-thunder-aixz34vy`. Schema changes go through `db:generate` → `db:migrate`; **never** `db:push`.

Known risks

- **Demo credentials in the repo.** `deployment.md` lists live demo passwords — including the platform super-admin account. They must be rotated or deleted before any real production use. See `deployment.md`.
- **Locale cookie gap** — cosmetic only; a fresh device shows English until the user re-picks (see `roadmap.md` open items).

Analytics & BI

Decision-oriented dashboards plus standardized data export, all currency-safe.

Dashboards

Surfaced on the agency dashboard and `/finance`:

- Revenue evolution (monthly)
- Top destinations / clients / markets by revenue
- Revenue per agent
- Conversion rate, average booking value, MoM growth
- Pipeline funnel **by value** + weighted forecast / win rate
- AR aging buckets
- Margin %
- Commission-by-supplier

Currency-safe by design

No FX conversion (see `business-rules.md`). Every metric **groups by currency** rather than summing across, with a **DZD headline**.

`src/lib/analytics.ts`

Pure helpers (no I/O), unit-testable:

- `sumByCurrency` — totals grouped by currency code
- `monthlyBuckets` — time-series bucketing
- `topN / countBy` — rankings and tallies
- `conversionRate, growthPct` — rate/growth metrics
- `agingBuckets` — AR aging

Charts render via `src/components/charts/` (recharts 3, tokenized), including `HBarInsight` and `FunnelInsight`.

BI export (`/reports`)

Gated by `canViewFinance` (admin/manager/finance). One-click CSV + Excel export of every core dataset, plus a full multi-sheet workbook.

- `src/lib/export/` — `csv.ts` (RFC-4180 + UTF-8 BOM, Power BI friendly), `xlsx.ts` (exceljs workbook), `datasets.ts` (BI dataset registry).
- Each enum field emits **both** `*_code` and `*_label` columns; amounts in DZD, dates in ISO; a date-range filter applies.

- Served by `api/export/[entity]` — CSV/XLSX per dataset, or `workbook` = all sheets.

AI Features

Built on the **Vercel AI SDK + OpenRouter** (`OPENROUTER_API_KEY`). All features degrade/no-op gracefully when the key is unset. See `api-integrations.md`.

AI must never

Hard guardrails for every AI feature. These extend the never rules to the AI surface:

- **Never invent prices** — quote only live supplier rates or stored values, never a guessed number.
- **Never invent bookings** — only create a booking when explicitly asked.
- **Never invent clients** — never fabricate a client record.
- **Never invent payments** — never record or imply a payment that didn't happen.
- **Never modify data without confirmation** — mutations are explicit and reviewable, not silent side effects of a chat reply.
- **Never hide errors** — surface failures to the user; don't paper over a failed tool call with a plausible answer.
- **Never perform destructive actions silently** — no deletes/overwrites without a clear, confirmed user action.

Current state: all assistant tools are agency-scoped. Of them only `createBooking` mutates — it creates a **draft** booking (zero balance, unconfirmed), and the system prompt forbids inventing supplier confirmations or PNRs and restricts creation to explicit requests. Everything else (`searchFlights`, `searchHotels`, `findClients`, `bookingsSummary`) is read-only. Gap: creation is intent-gated by the prompt, not yet behind a hard confirm step — the "modify without confirmation" guard is a convention, not an enforced gate.

AI assistant (`/assistant`)

Chat backed by `api/chat` with **agency-scoped tools**: find clients, bookings summary, create booking, search flights/hotels. Tools respect tenant scoping like every other action.

Inline features

Embedded where the work happens, not in a separate AI section:

- **AI itinerary generation** — one-click generation from booking items; saves to `booking_day` rows. Shareable via `/i/[token]`.
- **AI quote builder** — natural-language brief → structured proposal line items, pre-filled in the new-proposal form.
- **AI email drafting** — generate subject + body for confirmation, voucher, follow-up, or custom emails from the booking messages panel.
- **AI visa assistant** — per-nationality visa requirement summary from traveller passport

nationalities + booking destination.

Implementation

- Server actions in `src/lib/actions/ai.ts`.
- Itinerary helpers in `src/lib/itinerary.ts`.
- The assistant's tool definitions are agency-scoped wrappers over existing server actions, so AI cannot bypass security.md tenant isolation.